UiA University of Agder

Master

Reflect on Continuous Behaviour

Matin Mohammadi, Robert Hanssen, Sander Gnanavel

IKT467-1 25H

University of Agder, 2025
Faculty of Engineering and Science
Department of Engineering and Sciences

Obligatorisk gruppeerklæring

Den enkelte student er selv ansvarlig for å sette seg inn i hva som er lovlige hjelpemidler, retningslinjer for bruk av disse og regler om kildebruk. Erklæringen skal bevisstgjøre studentene på deres ansvar og hvilke konsekvenser fusk kan medføre. Manglende erklæring fritar ikke studentene fra sitt ansvar.

1.	Vi erklærer herved at vår besvarelse er vårt eget arbeid, og at vi ikke har brukt andre kilder eller har mottatt annen hjelp enn det som er nevnt i besvarelsen.	Ja / Nei
2.	**Vi erklærer videre at denne besvarelsen:** • Ikke har vært brukt til annen eksamen ved annen avdeling/universitet/høgskole innenlands eller utenlands. • Ikke refererer til andres arbeid uten at det er oppgitt. • Ikke refererer til eget tidligere arbeid uten at det er oppgitt. • Har alle referansene oppgitt i litteraturlisten. • Ikke er en kopi, duplikat eller avskrift av andres arbeid eller besvarelse.	Ja / Nei
3.	Vi er kjent med at brudd på ovennevnte er å betrakte som fusk og kan medføre annullering av eksamen og utestengelse fra universiteter og høgskoler i Norge, jf. Universitets- og høgskoleloven §§4-7 og 4-8 og Forskrift om eksamen §§ 31.	Ja / Nei
4.	Vi er kjent med at alle innleverte oppgaver kan bli plagiatkontrollert.	Ja / Nei
5.	Vi er kjent med at Universitetet i Agder vil behandle alle saker hvor det forligger mistanke om fusk etter høgskolens retningslinjer for behandling av saker om fusk.	Ja / Nei
6.	Vi har satt oss inn i regler og retningslinjer i bruk av kilder og referanser på biblioteket sine nettsider.	Ja / Nei
7.	Vi har i flertall blitt enige om at innsatsen innad i gruppen er merkbart forskjellig og ønsker dermed å vurderes individuelt. Ordinært vurderes alle deltakere i prosjektet samlet.	Ja / Nei

Publiseringsavtale

Fullmakt til elektronisk publisering av oppgaven Forfatter(ne) har opphavsrett til oppgaven. Det betyr blant annet enerett til å gjøre verket tilgjengelig for allmennheten (Åndsverkloven. §2).
Oppgaver som er unntatt offentlighet eller taushetsbelagt/konfidensiell vil ikke bli publisert.

Vi gir herved Universitetet i Agder en vederlagsfri rett til å gjøre oppgaven tilgjengelig for elektronisk publisering:	Ja / Nei
Er oppgaven båndlagt (konfidensiell)?	Ja / Nei
Er oppgaven unntatt offentlighet?	Ja / Nei

Contents

Chapter 1

Continuous changes

In the real highway system that our model refers to, many quantities change in a way that is naturally described as continuous. Vehicles do not jump from one position to another, and their speed does not switch between a finite set of discrete values. Instead, their motion emerges from a continuous adjustment of acceleration over time.

1.1 Continuously changing variables

At the microscopic level, the core continuous quantities are the kinematic attributes of each vehicle. In our Python implementation, these are collected in the `Kinematics` data class:

- `pos_m`: longitudinal position along the road in meters,
- `speed_mps`: speed in meters per second,
- `accel_mps2`: longitudinal acceleration in meters per second squared.

In the continuous system, each of these attributes can be viewed as a function of time. For a vehicle i on the road, we write:

$x_i(t) : \text{Time} \to \mathbb{R}$	position along the road
$v_i(t) : \text{Time} \to \mathbb{R}$	speed
$a_i(t) : \text{Time} \to \mathbb{R}$	acceleration

In free flow, $x_i(t)$ is a smooth, strictly increasing function of time as the vehicle progresses along the road. In congested conditions, the same function develops plateaus when the vehicle is stopped, but it never jumps backwards. Changes in $v_i(t)$ and $a_i(t)$ are also gradual. Even hard braking and strong acceleration take some time.
The implementation does not store the headway gap to the leading vehicle as a separate attribute, instead it is derived when needed from positions and lengths. If j is the vehicle directly ahead of i, with position $x_j(t)$ and length L_j, then the physical gap is

$$h_i(t) = x_j(t) - x_i(t) - L_j.$$

In reality, this gap changes continuously as both vehicles accelerate and brake. Even when a driver aims to maintain a constant following distance, what actually happens is a continuous adjustment process where $h_i(t)$ oscillates around a desired value. On top of the microscopic quantities, the model also uses macroscopic traffic variables such as density and flow, defined over spatial regions. For each `Vehicle`, the attributes `pos_m`, `speed_mps` and `accel_mps2` correspond to attribute functions $x_i(t)$, $v_i(t)$ and $a_i(t)$. For the `ObservationZone`, the attributes `zone_density_vpkm`, `zone_speed_mps` and `zone_flow_vph` correspond to attribute functions $k_Z(t)$, $u_Z(t)$ and $q_Z(t)$. The discrete simulator samples these functions at fixed time intervals, and together these functions form a continuous description of how the system evolves.

Chapter 2

Continuous behaviour

The continuous behaviour of a vehicle is defined by the fundamental laws of motion. These laws describe the relationships between the continuous variables identified in the previous chapter. The core of this behaviour is a system of first-order ordinary differential equations.

The first equation states that the rate of change of a vehicle's position is its speed. We express this using prime notation to represent the derivative with respect to time.

$$x_i'(t) = v_i(t)$$

The second equation states that the rate of change of a vehicle's speed is its acceleration.

$$v_i'(t) = a_i(t)$$

Together these two equations form the continuous system. The acceleration $a_i(t)$ is not described by its own differential equation but instead acts as the control input to this system. It is a value computed by the driver model at any given moment. This value is determined by a direct equation based on the driver's current discrete state, such as cruising or following, and its perception of the environment, such as the headway $h_i(t)$ and the relative speed to the vehicle in front.

This specification of behaviour, combining differential equations for the vehicle's physics with direct equations for the driver's decisions, provides a complete description of the continuous system. This formal description is what allows us to then create a discrete simulation model, for example by using the Euler method to approximate the solution to these differential equations at fixed time steps.

Chapter 3

Discretization

3.1 What information is lost

In a real road the state changes continuously. In the model we update only at fixed time points. Anything that happens between two time points is not captured. Exact event timing is rounded to the nearest tick. Short spikes in acceleration or braking that begin and end inside one step are smoothed out. Very short headway changes can be missed if they begin and end within one step. Controller delays for autonomous drivers are represented as an integer number of steps, so sub-step delay is lost. Lane merges or exits that would occur inside a step are applied at the next step, so the ordering may shift slightly in time.

3.2 Main sources of discretization error

Quantization of time shifts events by up to one half step. Forward Euler integration can overshoot when acceleration changes quickly. Sampling at step boundaries can under-report peaks in speed or acceleration. Aggregated metrics such as throughput per hour depend on counts per step and can show stair-step patterns. Delay and travel time are sums over steps and inherit the same rounding.

3.3 How we keep correctness

We choose a step size that is small compared to typical dynamics. The change in speed during one step is kept small relative to the speed limit. We enforce invariants at every step. Speed is clipped to the legal range. Position is clipped to the road bounds. Acceleration is clipped to the vehicle limits. We use a fixed update order for all agents to avoid race conditions. We run a warmup interval and start logging after warmup so initial transients do not bias results. For autonomous drivers we apply delay as a whole number of steps using the past state buffer. For events that must land inside a step we use sub-stepping locally. For example we can split one step into two half steps for collision checks or for a scenario trigger, then continue with the global step.

3.4 Recovering continuous signals for analysis

Plots may interpolate between samples to make curves smooth. KPIs can be computed with per-step values but integrated over windows that span many steps to reduce stair steps. If higher fidelity is needed for a metric we can export at a smaller step for that metric only while keeping the main simulation step for runtime.

3.5 Convergence and validation

We check that results converge as the step size is reduced. We run the same scenario with a smaller step and compare throughput, mean travel time, total delay, and density. Differences should fall below a set tolerance. We compare fundamental relationships such as the link between flow, speed, and density to a reference tool. If the smaller step changes the ordering of events but KPIs remain within tolerance, we accept the discretization.

3.6 Configuration guidance

Pick a step so that maximum acceleration times the step produces only a small change in speed. Keep the headway change within one step small relative to the desired gap. Use the same step when comparing scenarios so differences are due to the scenario and not the step. Log the step size together with every run so results are traceable.

Chapter 4

Time versus attribute types.

4.1 Two axes in the model

Time advances in discrete steps. Attributes are also stored in discrete form. Some attributes represent phenomena that are continuous in the real world. We sample them at each step. Other attributes are inherently discrete and change only when an event occurs.

4.2 Continuous-like attributes sampled in time

Position, speed, and acceleration evolve continuously in reality. In the model they hold one value per step. The driver logic proposes an acceleration. The engine integrates speed and position once per step. Precision is fixed. Position uses one tenth of a meter. Speed and acceleration use one hundredth in their own units. The chosen precision and the step size together define how finely we can see motion.

4.3 Discrete attributes updated at events

Identifiers and flags are exact and change rarely. The autonomous flag is true or false and selects the driver type. Driver mode in the state machine is a small set of labels such as *cruise* or *follow* or *hard brake*. These values change only when a guard condition becomes true. All such changes are applied at a time point so the model remains deterministic.

4.4 Static inputs versus dynamic state

Some attributes are inputs for the entire run. The simulation duration and the scenario mix are given at the start and do not change. Road geometry and speed limit are also fixed. Other attributes are dynamic. Vehicle kinematics and driver mode change at steps. Derived aggregates such as density and throughput are computed from the dynamic state after each step.

4.5 Interweaving of the two dimensions

Discrete time gates all changes. Even when a threshold is crossed within a step the state machine transition occurs at the next step. This keeps causality and ordering simple. The update order is the same for every step and for every vehicle. First we read the previous state. Then we decide. Then we apply limits. Then we integrate. Then we log. Discrete attributes can depend on sampled continuous values. A driver switches from cruise to follow when the sampled headway falls below the following threshold. The threshold check happens at a step and uses sampled values with fixed precision.

4.6 Latency and memory across steps

Autonomous drivers use a delay measured in seconds. We implement it as an integer number of steps. The controller reads signals from a buffer of past samples. This ties a continuous concept to discrete time through the step length. A smaller step makes the delay representation finer. A larger step makes it coarser.

4.7 Numerical choice and semantic choice

The time step controls how accurate the motion updates are. The decimal precision controls how values are stored and how they are compared to thresholds. The meaning of discrete attributes such as driver mode comes from how you define the labels and the rules that switch between them. We pick a time step and a precision that are fine enough so that a change of mode is not caused by rounding noise. When a mode depends on a threshold we add a small gap between enter and exit values. This prevents fast flipping back and forth due to tiny fluctuations.

4.8 Correctness under both views

We enforce legal ranges for continuous-like attributes after every update. We enforce invariants for discrete attributes at the same time. One vehicle has exactly one driver. Position stays within the road. Speed stays within limits. Mode switches follow their guards. Because both enforcement steps run on the same clock, the model stays consistent.

4.9 Logging and KPIs

We log sampled continuous values and discrete states at each step. KPIs are computed from these logs. Some KPIs are sums over steps such as total delay. Some are counts per step such as throughput. The combination of step size, precision, and event definitions determines KPI stability. We keep these settings fixed across scenarios so comparisons remain fair.

4.10 Reflection

The world is continuous but the simulation is not. Discrete time provides order and repeatability. Discrete attribute types provide clear semantics. Continuous-like quantities are sampled and bounded. Discrete quantities are switched by guards. The two dimensions work together at each step so the model is both tractable and faithful to the behaviours we study.

Chapter 5

Randomness

5.1 Where discretization creates randomness

Reducing a continuous world to discrete steps introduces several random-like effects even when inputs are fixed. Event timing is rounded to the nearest tick, which adds timing jitter. Short impulses in acceleration or braking are partially missed, which adds amplitude jitter. When two actions would happen inside the same step, the model resolves them at the next step, which adds ordering jitter. Sampling turns smooth signals into stepwise records, which adds quantization noise. Delays for autonomous control are mapped to a whole number of steps, which adds delay jitter.

5.2 How this interacts with real stochastic inputs

The model also contains true randomness from initial conditions, driver variation, arrivals, and tie breaking. Discretization noise and real randomness blend in the outputs. If the step is too large, discretization noise can dominate the natural spread and hide real effects. If the step is small, the natural spread from inputs dominates as intended.

5.3 Controls we apply

We fix a seed so that one run is exactly repeatable. We choose a step that keeps per-step speed change small compared to typical speeds. We use a fixed update order for all vehicles so tie breaking is consistent. We clip speed, position, and acceleration to legal bounds after every update so spikes cannot accumulate. We apply local sub-stepping for rare in-step events such as tight merges or scenario triggers, then return to the global step. We smooth plotted signals with simple interpolation only for display and never for state updates. We log the seed, the step, and all inputs with the outputs.

5.4 How we judge correctness under discretization

We run several seeds and report averages and spread for key measures such as throughput, mean travel time, total delay, and density. We repeat the same scenarios with a smaller step to check that results change only within a small tolerance. We compare aggregated relationships such as the link between flow, speed, and density to a trusted reference. If the smaller step shifts exact event times but the key measures stay within tolerance, we accept the discretization. If not, we reduce the step or add local sub-stepping at the point of error until the measures stabilize.

5.5 What remains random and why that is acceptable

Timing, amplitude, and ordering jitter at the scale of one step remain in the data. This is expected in a discrete model. The checks above ensure that this jitter stays smaller than the real variability we aim to study. As a result, the conclusions depend on stable statistics across runs, not on any one micro event.